

Semaine 3 | Module B

HTML5 — Formulaires & accessibilité

Anne Terrier — Développement web, classe passerelle

Cours 3h

TP 4h

Fil rouge MiniForum

Objectifs de la semaine

- Construire un formulaire complet avec les types d'input HTML5
- Distinguer le rôle de `name` (le serveur) de celui de `id` (la page)
- Utiliser la validation native du navigateur — et connaître ses limites
- Associer chaque champ à son `<label>` et grouper les champs liés
- Appliquer les principes de base de l'accessibilité : `alt`, contraste, navigation au clavier
- Contrôler une page avec le vérificateur d'accessibilité des outils de développement

Où en sommes-nous ?

La semaine 2 a servi à **afficher** du contenu. Cette semaine, on ouvre la page dans l'autre sens : le visiteur va pouvoir **saisir** quelque chose. C'est la dernière séance consacrée au HTML — le CSS commence en semaine 4.

Les formulaires écrits ici sont ceux du MiniForum final : leur structure ne changera plus jusqu'à la soutenance. Ils resteront inertes quelques semaines, le temps que JavaScript (semaine 7) puis le serveur (semaine 15) viennent les brancher.

1 À quoi sert un formulaire

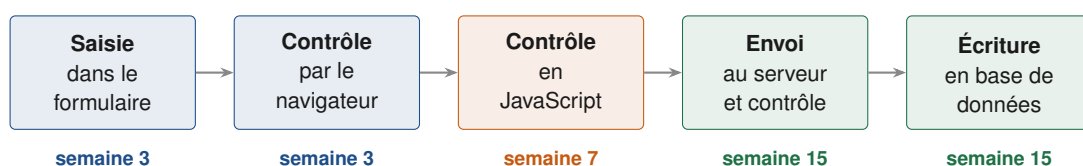
Jusqu'ici, le MiniForum ne fait que présenter des informations écrites en dur dans le HTML. Un formulaire inverse le sens de circulation : c'est le **point d'entrée** par lequel les données d'un utilisateur arrivent dans l'application.

Définition — Formulaire

Un **formulaire** est un ensemble de champs de saisie, regroupés dans une balise `<form>`, permettant à un visiteur de transmettre des données à l'application.

La vie d'une donnée saisie

Une donnée tapée par un visiteur traverse plusieurs étapes avant d'être enregistrée. Ce cours les construit une par une, sur toute l'année :



Cette semaine couvre les deux premières cases. Les trois autres expliquent pourquoi certains choix faits aujourd'hui — l'attribut `name` en particulier — paraîtront inutiles jusqu'en décembre.

La balise <form>

```
1 <form action="/topics" method="post">
2   <!-- les champs de saisie viennent ici -->
3   <button type="submit">Publier</button>
4 </form>
```

Extrait 1 – L’enveloppe d’un formulaire

Attribut	Rôle
action	L’adresse à laquelle les données sont envoyées
method	La façon de les envoyer : get (dans l’URL) ou post (dans le corps de la requête)

Ces deux attributs ne serviront qu’en semaine 15

Tant qu’aucun serveur n’écoute à l’adresse indiquée, `action` et `method` n’ont aucun effet visible : cliquer sur « Publier » recharge la page, et c’est tout.

Retiens seulement la logique : `get` place les données **dans l’URL** — pratique pour un formulaire de recherche que l’on veut pouvoir mettre en favori, inacceptable pour un mot de passe. `post` les place dans le corps de la requête, invisible dans la barre d’adresse. On y reviendra en détail avec Express.

2 Les champs de saisie

<input> et son attribut type

<input> est une balise auto-fermante dont le comportement change entièrement selon son attribut `type`.

type	Usage	Contrôle automatique
text	Texte libre sur une ligne	aucun
email	Adresse e-mail	présence d’un @ et d’un domaine
password	Mot de passe	aucun ; masque les caractères
number	Valeur numérique	chiffres uniquement, bornes min / max
date	Date	sélecteur de calendrier, format garanti
url	Adresse web	format d’URL
tel	Numéro de téléphone	aucun (les formats varient trop)
checkbox	Case à cocher	—
radio	Choix unique dans un groupe	—
hidden	Donnée transmise sans être affichée	—

Le bon type améliore surtout le téléphone

Sur mobile, le clavier affiché s’adapte au type du champ : pavé numérique pour `number`, clavier avec @ pour `email`, calendrier pour `date`. Choisir le bon type est le moyen le moins coûteux d’améliorer un formulaire.

Les autres champs

```

1 <!-- Texte long, sur plusieurs lignes -->
2 <textarea id="message" name="message" rows="6"
3     maxlength="500"></textarea>
4
5 <!-- Liste déroulante -->
6 <select id="categorie" name="categorie">
7     <option value="general">Général</option>
8     <option value="aide">Aide</option>
9     <option value="projets">Projets</option>
10 </select>
11
12 <button type="submit">Publier</button>

```

Extrait 2 – Zone de texte, liste déroulante et bouton

Balise	Rôle
<textarea>	Zone de texte multi-lignes. Jamais auto-fermante : son contenu initial se place entre les deux balises
<select> <option>	/ Liste déroulante. C'est l'attribut value de l'<option> qui est transmis, pas le texte affiché
<button>	Bouton. type="submit" envoie le formulaire, type="button" ne fait rien sans JavaScript

Le piège du <button> sans type

À l'intérieur d'un <form>, un <button> sans attribut type vaut submit par défaut : il envoie le formulaire et recharge la page. C'est la cause d'une bonne moitié des « mon bouton ne marche pas » de la semaine 7. Précise toujours le type.

Cases à cocher et boutons radio

Une **case à cocher** autorise plusieurs sélections indépendantes ; un **bouton radio** impose un choix unique dans un groupe.

```

1 <!-- Case à cocher : chaque case est indépendante -->
2 <input type="checkbox" id="news" name="news">
3 <label for="news">Recevoir la newsletter</label>
4
5 <!-- Boutons radio : même name = même groupe -->
6 <input type="radio" id="pub" name="visibilite" value="public" checked>
7 <label for="pub">Public</label>
8
9 <input type="radio" id="priv" name="visibilite" value="prive">
10 <label for="priv">Privé</label>

```

Extrait 3 – Checkbox et radio

La règle des boutons radio

Pour qu'un groupe de boutons radio fonctionne — un seul sélectionnable à la fois — tous ses boutons doivent partager le **même** name. C'est cet attribut commun qui les relie ; c'est value qui distingue l'option choisie, et id qui reste unique pour le <label>.

Deux name différents produisent deux groupes indépendants, chacun cochable séparément : symptôme classique quand « les deux boutons restent allumés ».

3 Les attributs qui encadrent la saisie

Attribut	Effet
name	Le nom sous lequel la valeur sera transmise au serveur. Sans lui, le champ n'est pas envoyé
id	Identifiant unique dans la page ; sert au <label>, au CSS et à JavaScript
value	La valeur initiale du champ (ou, pour radio et checkbox, la valeur envoyée)
placeholder	Texte d'exemple grisé, qui disparaît dès la première frappe
required	Le champ doit être rempli pour que le formulaire parte
maxlength	/ Bornent le nombre de caractères
minlength	
min / max / step	Bornent une valeur numérique ou une date
pattern	Impose un format via une expression régulière
readonly	Champ visible et transmis, mais non modifiable
disabled	Champ inactif et non transmis au serveur
autocomplete	Autorise le navigateur à proposer une valeur déjà saisie

name et id : deux mondes

C'est la confusion la plus tenace de la semaine. Les deux attributs se ressemblent, portent souvent la même valeur, et ne servent pourtant à rien de commun.

	id	name
Sert à	désigner l'élément dans la page	nommer la valeur pour le serveur
Utilisé par	le <label>, le CSS, JavaScript	le formulaire au moment de l'envoi
Unicité	unique dans la page	partagé par les boutons radio d'un même groupe
Si absent	impossible de relier un label	le champ n'est pas transmis

Aperçu — ce que le serveur recevra en semaine 15

Au moment de l'envoi, le navigateur assemble les champs en une suite de paires nom=valeur. Pour le formulaire de création de sujet, cela donne :

```
pseudo=alice&titre=Bonjour&categorie=general&message=Salut
```

Chaque clé de cette liste est un attribut name. En semaine 15, le serveur Express lira exactement ces clés : req.body.pseudo, req.body.titre... Un champ sans name sera tout simplement absent de la liste, et le serveur ne saura pas qu'il a existé.

C'est pour cette raison qu'on nomme les champs **dès maintenant**, alors même que rien ne les lit encore.

pattern : imposer un format

```

1 <label for="pseudo">Pseudo</label>
2 <input type="text" id="pseudo" name="pseudo"
3     pattern="[A-Za-z0-9]{3,15}"
4     title="3 à 15 caractères : lettres et chiffres uniquement"
5     required>
```

Extrait 4 – Un pseudo de 3 à 15 caractères, lettres et chiffres seulement

pattern sans title est inutilisable

Si la saisie ne correspond pas au motif, le navigateur affiche un message d'erreur générique du type « Veuillez respecter le format demandé » — l'utilisateur ne sait pas quoi corriger.

L'attribut `title` fournit le texte affiché à la place. Il n'est pas décoratif : sans lui, le champ devient un piège.

4 La validation native et ses limites

Le navigateur contrôle lui-même certains champs avant d'envoyer le formulaire, sans une ligne de JavaScript. Un champ `required` vide ou un `type="email"` mal formé bloquent l'envoi et déclenchent une bulle d'avertissement.

Tester la validation

Laisse un champ `required` vide et clique sur le bouton d'envoi : le navigateur doit refuser, placer le curseur dans le champ fautif et afficher un message. C'est le premier test à faire sur chaque formulaire du TP.

Ce qu'elle apporte

- Un retour **immédiat**, avant tout aller-retour avec le serveur
- Des messages traduits dans la langue du navigateur, sans effort
- Un premier filtre contre les saisies manifestement erronées

Ce qu'elle ne fait pas

Une validation côté navigateur ne protège rien

La validation HTML5 est une **aide à la saisie**, pas une sécurité. Elle s'exécute sur la machine du visiteur, qui garde la main dessus.

Il suffit d'ouvrir les outils de développement et de supprimer l'attribut `required` pour envoyer un formulaire vide. Il suffit même de ne pas utiliser de navigateur du tout : n'importe quel outil en ligne de commande peut envoyer une requête directement au serveur, sans jamais avoir vu le formulaire.

Conséquence, valable pour toute l'année : tout contrôle effectué dans le navigateur doit être **refait sur le serveur**. On le mettra en œuvre en semaine 15, et on verra en semaine 19 ce qu'un attaquant fait d'un serveur qui a fait confiance au navigateur.

Niveau de contrôle	Ce qu'il assure	Vu en
Navigateur (HTML5)	Confort de saisie, erreurs évidentes	semaine 3
JavaScript (client)	Messages sur mesure, règles combinées	semaine 7
Serveur (Express)	La seule barrière fiable	semaine 15
Base de données	Intégrité des données stockées	semaine 8

5 Structurer un formulaire

Un formulaire de plus de trois ou quatre champs a besoin d'être organisé — pour l'œil comme pour les lecteurs d'écran.

```
1 <form action="/topics" method="post">
```

```

2  <fieldset>
3    <legend>Votre message</legend>
4
5    <label for="titre">Titre du sujet</label>
6    <input type="text" id="titre" name="titre"
7          maxlength="80" required>
8
9
10   <label for="message">Message</label>
11   <textarea id="message" name="message"
12           rows="6" maxlength="500" required></textarea>
13 </fieldset>
14
15 <fieldset>
16   <legend>Visibilité</legend>
17
18   <input type="radio" id="pub" name="visibilite"
19         value="public" checked>
20   <label for="pub">Public</label>
21
22   <input type="radio" id="priv" name="visibilite" value="prive">
23   <label for="priv">Réservé aux membres</label>
24 </fieldset>
25
26 <button type="submit">Publier</button>
27 </form>

```

Extrait 5 – Regrouper des champs liés

Balise	Rôle
<fieldset>	Regroupe des champs qui vont ensemble
<legend>	Le titre du groupe ; première balise à l'intérieur du <fieldset>

<fieldset> est indispensable pour un groupe de boutons radio

Un lecteur d'écran annonce l'étiquette de chaque bouton radio — « Public », « Réservé aux membres » — mais rien ne dit *de quoi* il s'agit. Le <legend> fournit cette information manquante : la personne entend « Visibilité : Public ».

Pour un champ isolé, le <label> suffit. Pour un groupe, il faut les deux.

6 Accessibilité

Définition — Accessibilité (a11y)

L'**accessibilité** consiste à rendre un site utilisable par tous, y compris les personnes en situation de handicap : malvoyance, cécité, daltonisme, difficultés motrices empêchant l'usage d'une souris, troubles cognitifs.

Ce n'est pas une couche que l'on ajoute à la fin : elle se joue à 80 % dans le choix des balises HTML, donc maintenant.

Trois raisons de s'en préoccuper dès la troisième semaine :

- **Des gens.** Une page inaccessible exclut des utilisateurs réels, souvent sans que personne s'en aperçoive.

- **La loi.** Les sites publics suisses sont soumis à des exigences d'accessibilité, et la plupart des appels d'offres publics les imposent.
- **La qualité.** Un HTML accessible est un HTML bien structuré : il est aussi mieux référencé et plus facile à maintenir.

<label> : relier une étiquette à son champ

C'est le point le plus important de la semaine. Un champ sans `<label>` correctement relié est, pour un lecteur d'écran, une case vide dont personne ne sait quoi faire.

```

1 <!-- Écriture explicite : for = id (la plus courante) -->
2 <label for="pseudo">Votre pseudo</label>
3 <input type="text" id="pseudo" name="pseudo">
4
5 <!-- Écriture implicite : le label enveloppe le champ -->
6 <label>
7   Votre pseudo
8   <input type="text" name="pseudo">
9 </label>
```

Extrait 6 – Deux écritures possibles, un seul résultat

for doit reprendre id, pas name

Un `for` qui pointe vers un `id` inexistant ne relie rien du tout — et le validateur W3C le signale. Vérifie le lien en **cliquant sur l'étiquette** dans le navigateur : si le curseur se place dans le champ (ou si la case se coche), la liaison fonctionne. C'est un test d'une seconde, à faire sur chaque champ.

Un placeholder n'est pas un <label>

Il est tentant de supprimer les étiquettes et de tout mettre dans le placeholder : le formulaire paraît plus épuré. C'est une mauvaise idée pour quatre raisons :

- le texte **disparaît** dès la première frappe, et l'utilisateur ne sait plus ce qu'il remplit ;
- son contraste est volontairement faible, donc difficile à lire ;
- plusieurs lecteurs d'écran ne l'annoncent pas ;
- il n'y a plus de zone cliquable pour atteindre le champ.

Le placeholder donne un **exemple** de saisie (ex : `alice`) ; le `<label>` dit ce qu'est le champ. Les deux, pas l'un à la place de l'autre.

Le texte alternatif des images

Rappel de la semaine 2, complété par la règle qui manquait : **le bon alt dépend du rôle de l'image.**

Rôle de l'image	Que mettre dans alt
Elle porte une information	Ce que l'image dit : <code>alt="Trois sujets sans réponse depuis lundi"</code>
Elle est un lien ou un bouton	La destination ou l'action : <code>alt="Retour à l'accueil"</code>
Elle est décorative	Un alt vide : <code>alt=""</code> , pour que le lecteur d'écran l'ignore

Le test du téléphone

Imagine que tu décrives la page à quelqu'un au téléphone. Ce que tu dirais de l'image, c'est le `alt`. Ce que tu passerais sous silence parce que c'est un simple élément de décor, c'est un `alt=""`.

Le contraste des couleurs

Un texte gris clair sur fond blanc est illisible pour beaucoup de gens — et pour tout le monde en plein soleil.

Définition — Rapport de contraste

Le **rapport de contraste** compare la luminosité d'un texte et de son fond. Il va de 1 :1 (invisible) à 21 :1 (noir sur blanc).

Le seuil de référence est le niveau **AA** des règles WCAG : **4,5 :1** pour le texte courant, **3 :1** pour les grands titres.

La couleur ne doit jamais être la seule information

Un champ en erreur signalé **uniquement** par une bordure rouge est invisible pour une personne daltonienne — environ 8 % des hommes.

Double systématiquement la couleur d'un second signal : un texte (« Ce champ est obligatoire »), une icône, ou un soulignement. La règle vaut aussi pour les liens dans un paragraphe : ils doivent rester reconnaissables sans la couleur.

La navigation au clavier

Toutes les personnes n'utilisent pas de souris : handicap moteur, lecteur d'écran, ou simple habitude. Un site doit rester utilisable au seul clavier.

Touche	Action
Tab	Passer à l'élément interactif suivant
Maj + Tab	Revenir au précédent
Entrée	Activer un lien ou envoyer un formulaire
Espace	Cocher une case, activer un bouton
Flèches	Se déplacer dans un groupe de boutons radio ou une liste déroulante

Le test des trente secondes

Ouvre ta page, clique dans la barre d'adresse, puis parcours toute la page avec Tab sans jamais toucher la souris. Trois choses doivent être vraies :

- tu **vois toujours** où tu te trouves (un cadre entoure l'élément actif) ;
- l'ordre suit la **lecture naturelle** de la page, de haut en bas ;
- tu peux atteindre **tous** les liens, champs et boutons, et envoyer le formulaire.

Deux règles à retenir pour la suite

L'ordre de tabulation suit l'ordre du code, pas l'ordre visuel. Si le CSS déplace un élément (semaines 4 et 5), la navigation clavier suivra toujours le HTML : c'est une raison de plus pour écrire le HTML dans un ordre logique dès maintenant.

Ne supprime jamais le cadre de focus. On trouve souvent `outline: none` ; dans du CSS trouvé en ligne : cette seule ligne rend un site inutilisable au clavier. Si le cadre par défaut ne plaît pas, on le **remplace** par un autre style visible — on ne l'efface pas.

Vérifier son travail

Outil	Ce qu'il apporte
Onglet <i>Accessibilité</i> des outils de développement	Affiche l'arbre tel que le perçoit un lecteur d'écran, et signale les contrastes insuffisants
Lighthouse (Chrome, onglet du même nom)	Produit un rapport noté, avec la liste des problèmes détectés
validator.w3.org	Repère déjà les alt manquants, les for orphelins et les id en double
Le clavier	Le test le plus rapide, et celui qu'aucun outil ne remplace

Aucun outil automatique ne suffit

Un rapport Lighthouse à 100 ne signifie pas qu'une page est accessible : une machine sait vérifier qu'un alt existe, pas qu'il décrit correctement l'image. Ces outils repèrent les erreurs mécaniques ; le reste relève du jugement.

Mémo — les huit réflexes de la semaine

- Chaque `<input>` a un `<label>` relié par `for` / `id`
- Chaque `<img>` a un alt adapté à son rôle (vide si décorative)
- Chaque champ à transmettre a un `name`
- Les groupes de boutons radio sont dans un `<fieldset>` avec `<legend>`
- Le placeholder complète le label, il ne le remplace pas
- L'information n'est jamais portée par la seule couleur
- La page se parcourt entièrement au clavier, avec un focus visible
- La page passe le validateur W3C sans erreur

7 Lexique de la semaine

Terme	Définition
Formulaire	Ensemble de champs permettant à un visiteur de transmettre des données
<code>name</code>	Nom sous lequel la valeur d'un champ est transmise au serveur
<code>value</code>	Valeur d'un champ ; pour un <code>radio</code> , l'option qu'il représente
<code>placeholder</code>	Exemple de saisie affiché dans un champ vide
<code>required</code>	Attribut rendant un champ obligatoire
<code>pattern</code>	Format imposé à la saisie, exprimé par une expression régulière
Validation native	Contrôle automatique effectué par le navigateur avant l'envoi
<code><fieldset></code> / <code><legend></code>	Groupe de champs liés et son titre
Accessibilité (a11y)	Ensemble des pratiques rendant un site utilisable par tous
Lecteur d'écran	Logiciel qui restitue une page à voix haute
Rapport de contraste	Mesure de lisibilité d'un texte sur son fond (seuil AA : 4,5 :1)
Focus	Élément qui reçoit actuellement les entrées clavier
Ordre de tabulation	Ordre dans lequel Tab parcourt les éléments interactifs
WCAG	Règles internationales d'accessibilité des contenus web

Résumé de la semaine

Ce qu'il faut retenir

- Un `<form>` contient des champs : `<input>`, `<textarea>`, `<select>`, `<button>`
- Le type d'un `<input>` change son comportement, son clavier mobile et son contrôle automatique
- `name` nomme la valeur **pour le serveur** ; `id` désigne l'élément **dans la page**
- Les boutons radio d'un même groupe partagent le même `name`
- La validation native est un **confort de saisie**, jamais une sécurité : tout sera revérifié côté serveur
- Chaque champ a un `<label>` relié ; les groupes ont un `<fieldset>` et une `<legend>`
- L'accessibilité repose sur le HTML : `alt` adapté, contraste suffisant, information jamais portée par la seule couleur
- Une page doit être entièrement utilisable **au clavier**, avec un focus visible

Fil rouge — MiniForum : tous les formulaires en place

Le MiniForum contient désormais l'intégralité de ses formulaires : création de sujet, réponse à un sujet, inscription et connexion.

Ils ne font encore rien — mais leur structure ne changera plus jusqu'à la fin du cours. Les `name` choisis cette semaine sont ceux que le serveur Express lira en semaine 15, et les `id` sont ceux que JavaScript utilisera dès la semaine 7.

Pour la semaine 4

- Les quatre formulaires du MiniForum terminés et poussés sur GitHub
- Chaque page passe `validator.w3.org` **sans erreur**
- Le site est entièrement parcourable au clavier, et le vérificateur d'accessibilité des outils de développement ne signale plus de problème bloquant
- La semaine 4 ouvre le **CSS** : la structure construite pendant trois semaines va enfin recevoir sa mise en forme

Le détail pas à pas se trouve dans la fiche *S03_TP*.